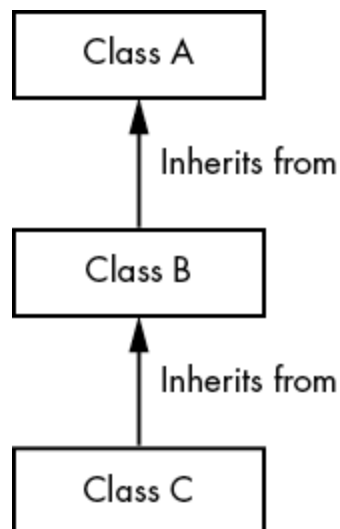


The two classes share everything other than the `__init__()` method. Therefore, the way that the buttons function, and the method calls that are available (`handleEvent()`, `draw()`, `disable()`, `enable()`, and so on), must be identical.

There are a number of benefits to this kind of inheritance. First, it provides consistency for both the client code and the end user: `TextButton` and `CustomButton` objects work the same way. It also makes bugs easier to fix—fixing a bug in a base class means you have then fixed the bug in all subclasses that inherit from it. Finally, if you add functionality in the base class, it is available immediately in all classes that inherit from the base class.

Class Hierarchy

Any class can be used as a base class, even a subclass that already inherits from another base class. This kind of relationship, known as a *class hierarchy*, is depicted in [Figure 10-9](#).



[Figure 10-9](#): A class hierarchy

In this figure, class `c` inherits from class `B`, which inherits from class `A`. Therefore, class `c` is a subclass and class `B` is a base class, but class `B` is also a subclass of class `A`. So, class `B` serves in both roles. In cases like this, class `c` inherits not only all the methods and instance variables in class `B`, but also